# CyberTrace Final Project Report

BCSE302P Database Systems Lab | 11 October 2026

## Project summary

CyberTrace is a local graph database application for exploring cyber threat intelligence. The delivered system uses Neo4j 5.x, a Flask API, and a responsive browser interface. A pinned MITRE ATT&CK Enterprise v19.2 STIX subset supplies documented threat actors, software, techniques, and `USES` relationships. A separate deterministic synthetic layer supplies organizations, vulnerabilities, documentation-range IP addresses, example domains, and scenario links. Each node and relationship exposes its own provenance so mixed paths are not mistaken for wholly documented incidents.

The final live verification on 11 October 2026 passed 27 automated tests and checked the loaded graph, CRUD, ten Cypher investigations, bounded shortest path, directed PageRank, and the real-record write guard. The baseline graph contains 159 nodes and 1,116 relationships.

## Problem and objectives

Flat lists make multi-step relationships between an actor, software, technique, infrastructure, and target difficult to inspect. CyberTrace's graph model allows relationship traversal while keeping the source of each claim visible. The objectives were to implement a Neo4j schema with sufficient label, relationship, and node coverage; load reproducible sourced and synthetic data; expose database operations and meaningful queries; provide an interactive interface; and demonstrate two graph analytics for the final review.

## Architecture and graph model

The browser calls JSON routes in Flask. The backend validates entity labels, mutable fields, relationship patterns, hop limits, and query limits before building Cypher. Values are passed as parameters. Neo4j driver-managed read and write transactions execute every database operation. Local administrator authentication uses a scrypt hash in ignored environment configuration, a same-site session cookie, and CSRF tokens on state-changing API calls. The development server binds to loopback by default.

Seven labels identify `ThreatActor`, `Malware`, `Technique`, `Vulnerability`, `IP`, `Domain`, and `Organization`. Six distinct relationship types are `USES`, `TARGETS`, `EXPLOITS`, `COMMUNICATES_WITH`, `RESOLVES_TO`, and `AFFECTS`. Nine directed label-to-label patterns are allowed by the management API. Stable IDs have uniqueness constraints; six fields have explicit search indexes. The graph's baseline has seven uniqueness constraints and six explicit search indexes, plus seven constraint-backed indexes.

## Dataset and provenance

The real source is the MITRE ATT&CK Enterprise v19.2 STIX release (https://github.com/mitre-attack/attack-stix-data/releases/tag/v19.2). `data/source-manifest.json` records the URL, release, retrieval timestamp, 53,835,637-byte source size, and SHA-256 checksum. Import code selects 25 groups, 40 software records, and 40 techniques, yielding 105 real nodes and 1,038 documented `USES` relationships. Their properties retain source identifiers, ATT&CK URLs, dataset version, and `is_synthetic=false`.

The deterministic synthetic generator adds 54 nodes and 78 relationships using seed 302. It uses only RFC 5737 documentation IP ranges and IANA example domains. Generated and manually created records have `source= Synthetic`, `is_synthetic=true`, and generator metadata. Synthetic `AFFECTS` links fill a sixth Neo4j relationship type but are not claimed as MITRE assertions. The importer and loader use stable identifiers and `MERGE`, while pre-load validation rejects missing endpoints, duplicates, unsafe infrastructure, and provenance violations. MITRE's terms of use (https://attack.mitre.org/resources/legal-and-branding/terms-of-use/) govern the ATT&CK-derived content; the copyright designation appears in `THIRD_PARTY_NOTICES.md`.

## Functions and queries

The UI provides live counts, search and full provenance details, bounded neighborhood visualization with a textual list, generic synthetic entity management, relationship creation/deletion, a three-step investigation view, shortest path, and PageRank. Real imported nodes and relationships are read-only through the API. The backend exposes ten named parameterized Cypher investigations: actor malware, malware techniques, actor targets, malware vulnerabilities, malware infrastructure, actor investigation, actor techniques, actor vulnerabilities, vulnerability organizations, and most connected entities. The loaded graph supports 90 complete actor-to-malware-to-IP-to-domain paths; the infrastructure steps are synthetic.

Shortest path accepts two allowlisted labels and stable IDs and searches up to six relationships in either traversal direction. It distinguishes a missing endpoint from existing endpoints without a path. The response contains ordered nodes, stored relationship direction, properties, and provenance. PageRank uses the directed stored graph with damping factor 0.85, dangling-node redistribution, convergence tolerance `1e-10`, and a 100-iteration cap. The user chooses real-only or all-data scope. This score expresses graph prominence only, not danger or severity.

## Verification against the review stages

Review 1 established the problem, graph choice, data source, schema, and local administrator requirement. Review 2 required an implemented database, CRUD operations, and advanced NoSQL features; the live system demonstrates these through Neo4j schema verification, CRUD scripts, ten investigations, multi-hop paths, and the graph UI. Review 3 emphasizes a working demonstration, advanced queries or analytics, usability, and viva readiness; the delivered interface includes shortest path and PageRank and the repository includes a user manual and demonstration notes.

The final command `python -m scripts.verify_final` passed 27 automated tests, data validation, 159-node and 1,116-relationship live checks, schema checks, CRUD, all ten investigations, generic entity and relationship operations, read-only enforcement for APT28, and both analytics. The real-only PageRank run used 105 nodes and 1,038 relationships and converged in 26 iterations; the all-data run used 159 nodes and 1,116 relationships and converged in 23 iterations. See `docs/final-verification.md` for the observed record and `PROJECT_TASKS.md` for task status.

## Reproducibility and submitted artifacts

The committed processed subset and synthetic fixture allow a fresh Neo4j instance to load the graph without repeating the large upstream download. To reproduce the selection itself, the source manifest specifies the exact v19.2 STIX URL, source size, and checksum. The importer verifies that checksum before transforming the source. `scripts.validate_data` checks graph coverage, endpoint resolution, provenance, and documentation-safe infrastructure before `scripts.load_neo4j` creates constraints and loads nodes and relationships. Running the loader again targets the same stable identifiers, so it does not duplicate fixture records.

The repository includes the Flask API, static frontend, schema and data pipeline, ten named Cypher investigations, analytics implementation, automated tests, verification scripts, user manual, final demonstration slides, and viva notes. The ignored `.env` holds local database and administrator secrets, while ignored `.runtime` holds the machine-specific Neo4j and Java binaries. A fresh clone needs Python dependencies, a reachable compatible Neo4j instance, and newly generated local administrator credentials. The README documents this setup and the one-command verification sequence.

For the live review, sign in and show the dashboard provenance split before opening APT28's sourced record. The graph list provides a text alternative to the SVG. Demonstrate a disposable synthetic CRUD record, then explain the synthetic portion of an actor-to-domain investigation. Finish with the one-hop synthetic `TARGETS` example and PageRank's real-only versus all-data scope. This order keeps data origin visible while showing the database operations and analytics required by the rubric.

## Limitations and responsible interpretation

The real dataset is a deliberately selected ATT&CK subset, not a complete threat feed. Synthetic scenario links enable lab traversal and do not establish real-world targeting or infrastructure. Shortest path treats relationships as traversable in either direction, whereas PageRank respects stored direction. PageRank can amplify selection and degree effects and should not be used as a threat score. The SVG is a bounded neighborhood view, not a full graph layout. The local single-user login and Flask development server are for a lab machine, not public deployment. The project-local Neo4j runtime is ignored; a fresh clone needs a compatible Neo4j installation and credentials.

## References


1. MITRE ATT&CK Enterprise STIX v19.2 release: https://github.com/mitre-attack/attack-stix-data/releases/tag/v19.2

2. MITRE ATT&CK terms of use: https://attack.mitre.org/resources/legal-and-branding/terms-of-use/

3. CyberTrace source manifest: `data/source-manifest.json`.

4. CyberTrace implementation and verification: `backend/`, `scripts/verify_final.py`, and `docs/final-verification.md`.